BFF Setup and Auth Guide

app-bff — Spring Boot API Gateway with Native JWT Auth & JDBC DAL

Author: R. Seeds

Status: Verified against live project source as of 2026-08-28

About This Document

This guide consolidates three planning documents used to build the app-bff project:

- Spring Boot API Gateway Data Scaffold for JWT Auth.docx
- Spring Boot API JDBC and Auth.docx
- Securing Spring API with HTTP TLS.docx

Each was cross-checked line-by-line against the actual pom.xml, application.properties, every Java source file, and app_db_init.sql in the project. Several steps in the original planning docs did not match what the working project actually required — those gaps are marked with a ⚠ **Correction** callout at the point they occur, and are also collected in the Troubleshooting & Gotchas Reference for quick lookup. Nothing in this document changes project source code — it documents the setup and configuration process as it actually exists today.

Full source for every class, config, and settings file is reproduced in Appendix A, one file per page, with a key mapping each entry back to the section that discusses it. The guide body below references those files rather than reprinting them inline.

Table of Contents

1. Project Initialization (start.spring.io)

The project as actually generated uses:

Setting	Value
Build tool	Maven
Language	Java
Java version	26
Spring Boot version	4.1.1 (parent)
Packaging	Jar
Group	org.apprenti
Artifact	app-bff

> **⚠ Correction**
> Spring Boot API JDBC and Auth.docx specified Spring Boot 3.x and Java 17/21 (LTS). The project as built targets Spring Boot 4.1.1 / Java 26 instead. This matters beyond version numbers — Spring Boot 4's dependency modularization renamed `spring-boot-starter-web` to **spring-boot-starter-webmvc** (see dependency table below). Following the original doc's dependency name verbatim on Boot 4 would not resolve.

Dependencies actually added (from pom.xml, Appendix A.1)

Dependency	Purpose
spring-boot-starter-jdbc	DataSource + JdbcClient support
spring-boot-starter-security	Core Spring Security filter chain, DaoAuthenticationProvider, JdbcUserDetailsManager
spring-boot-starter-security-oauth2-resource-server	JWT decoding/validation via Nimbus JOSE; brings in nimbus-jose-jwt transitively
spring-boot-starter-webmvc	REST controllers + embedded Tomcat (Boot 4 name for what used to be -web)
postgresql (runtime)	JDBC driver for PostgreSQL
*-test starters (jdbc, security, oauth2-resource-server, webmvc)	Test-scope counterparts, auto-added by Initializr

JWT library decision: Spring Boot API JDBC and Auth.docx presented two options — a manual JJWT (io.jsonwebtoken) implementation, or Spring's built-in OAuth2 Resource Server DSL backed by Nimbus JOSE. **Option B (Nimbus JOSE) was the one actually implemented.** JJWT is not a dependency in this project and none of its APIs appear in source — the JJWT section of that planning doc does not apply here.

2. Database Setup (PostgreSQL)

> **⚠ Correction**
> Both Spring Boot API JDBC and Auth.docx and Securing Spring API with HTTP TLS.docx assume schema creation happens through Spring Boot's automatic schema.sql initializer (the TLS doc even sets spring.sql.init.mode=always for this). The project does not use that mechanism at all. There is no schema.sql anywhere in src/main/resources, and the current application.properties (Appendix A.2) does not set spring.sql.init.mode. Schema setup is instead handled entirely by a standalone script run by hand.

The actual process:

- The database and schema are defined in app_db/app_db_init.sql (Appendix A.3), which connects to the postgres admin database, drops and recreates the app_db database, connects to app_db, creates the Spring Security schema (users, authorities + unique index) required by JdbcUserDetailsManager, and creates the domain schema (notes) used by the note-taking endpoints.
- Run it once, from the command line, against a running local PostgreSQL server:

```
psql -U postgres -f app_db_init.sql
```

- Confirm the datasource block in application.properties points at the database this script created:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/app_db
spring.datasource.username=postgres
spring.datasource.password=postgres1
spring.datasource.driver-class-name=org.postgresql.Driver
```

If you ever revisit this project and add tables to the notes domain schema (or any other schema change), re-run app_db_init.sql by hand — nothing in the running application will pick up schema changes automatically.

3. RSA Keypair for JWT Signing

Defined in RsaKeyConfig.java (Appendix A.5). This is a separate, unrelated RSA keypair from the TLS keystore covered in Section 4 — easy to conflate since both are "RSA keys," but they serve entirely different purposes:

	This keypair (RsaKeyConfig)	TLS keystore (keystore.p12)
Purpose	Signs/verifies JWTs (application-layer auth)	Encrypts the HTTPS wire (transport layer)
Generated	In memory, at application startup	Once, ahead of time, via mkcert
Persisted?	No — regenerated fresh on every restart	Yes — a file on disk (keystore.p12)

Three beans are produced: a 2048-bit KeyPair, and the derived RSAPublicKey / RSAPrivateKey. SecurityConfig wires the private key into a JwtEncoder (for signing tokens at login) and the public key into a JwtDecoder (for verifying tokens on every authenticated request) — see Section 5.

Operational note: because the keypair is regenerated in memory on every startup rather than persisted, any JWT issued before a restart fails verification after the app restarts (the public key used to verify it no longer matches). This is expected behavior for this scaffold, not a bug — just something to remember if a previously-issued token suddenly stops working after a redeploy.

4. TLS/HTTPS Keystore Setup (mkcert)

This section documents the verified, working process — not the version originally written in Securing Spring API with HTTP TLS.docx, which contains an incorrect assumption (flagged below).

4.1 Install mkcert

```
winget install FiloSottile.mkcert
```

Confirmed via winget show FiloSottile.mkcert: publisher is Filippo Valsorda (FiloSottile), installer pulled directly from the official github.com/FiloSottile/mkcert releases with a pinned SHA256 hash. Chocolatey (choco install mkcert) and Scoop (scoop install mkcert) are equally valid alternatives, as originally documented.

> **⚠ Gotcha**
>
> After installing, mkcert may not resolve in a terminal session that was already open before the install (PATH is updated via the registry, and existing sessions don't pick it up). Open a fresh terminal if you get CommandNotFoundException.

4.2 Install the local Certificate Authority

```
mkcert -install
```

4.3 Generate the keystore

```
mkcert -pkcs12 -p12-file keystore.p12 localhost 127.0.0.1 ::1
```

Run this from the directory you want the file to land in — mkcert writes to the current working directory, and a bare filename with no path is easy to lose track of in a long terminal session.

> **⚠ Correction**
>
> Securing Spring API with HTTP TLS.docx states that mkcert "reads the KEYSTORE_PASSWORD environment variable when bundling certificates into a PKCS12 file," and shows setting $env:KEYSTORE_PASSWORD before running the command. This does not work. Regardless of the environment variable, mkcert hardcodes the PKCS12 store password to the literal string changeit — mkcert's own CLI output says as much:
>
> `The legacy PKCS#12 encryption password is the often hardcoded default "changeit"`
>
> There is no mkcert flag or environment variable that produces a custom PKCS12 password directly.

4.4 Re-key the store to the actual target password

Since mkcert always produces a changeit-protected file, use keytool to re-encrypt it with the real password after generation:

```
keytool -storepasswd -new "#FSISeedsSWDJune2026" -keystore keystore.p12 -storetype PKCS12 -storepass changeit
```

This must be run against every copy of the file that's actually loaded by the application — specifically src/main/resources/keystore.p12 (the source of truth). Do not run it only against a stray copy left in your home directory from an earlier test run.

Verify it took, on both the store and the private key entry inside it:

```
keytool -list -v -keystore keystore.p12 -storetype PKCS12 -storepass "#FSISeedsSWDJune2026"
```

A successful listing shows "Entry type: PrivateKeyEntry" under the new password. (Some JDK versions only re-encrypt the outer store and leave the key entry itself on the old password — always confirm with -list -v, not just a silent exit code, before trusting the change.)

4.5 Note the alias

```
Alias name: 1
```

> **⚠ Correction**
>
> Securing Spring API with HTTP TLS.docx sets server.ssl.key-alias=localhost, assuming the certificate's alias matches the hostname it was issued for. mkcert always names the PKCS12 entry alias "1", never the hostname. The live application.properties correctly uses:
>
> `server.ssl.key-alias=1`
>
> If this keystore is ever regenerated, re-verify the alias with keytool -list rather than assuming it stayed 1.

4.6 Place the file and rebuild

Copy the finished, re-keyed keystore.p12 to src/main/resources/keystore.p12 (must be on the classpath — application.properties references it as classpath:keystore.p12).

> **⚠ Gotcha**
>
> Maven copies src/main/resources/** into target/classes/** as part of a build. If you edit or replace keystore.p12 (or any resource file) without triggering a real rebuild, the running app may still load the stale copy in target/classes, silently reproducing a bug you thought you already fixed. Always rebuild (mvn compile, or IntelliJ's Build/Rebuild) after changing a resource file — mvn clean compile if you want zero ambiguity.

4.7 Final SSL block in application.properties

```
server.port=8443
server.ssl.enabled=true
server.ssl.key-store-type=PKCS12
server.ssl.key-store=classpath:keystore.p12
server.ssl.key-store-password=#FSISeedsSWDJune2026
server.ssl.key-alias=1
```

4.8 Flagged for future revisit — HTTPS enforcement not implemented

Securing Spring API with HTTP TLS.docx, Step 4, includes this line in its SecurityFilterChain example:

```
.requiresChannel(channel -> channel.anyRequest().requiresSecure())
```

This forces all traffic onto HTTPS at the filter-chain level (rejecting/redirecting plain HTTP). **It is not present in the live SecurityConfig.java** (Appendix A.6) — the filter chain currently has no requiresChannel(...) call at all. This is being left as-is intentionally for now, flagged here for a later decision on whether to add it, rather than being silently applied or silently dropped from the record.

5. Security Configuration

Defined in SecurityConfig.java (Appendix A.6). Six beans:

Bean	Role
securityFilterChain	Wires CSRF (disabled), authorization rules, stateless sessions, and the OAuth2 resource server JWT filter
userDetailsManager	JdbcUserDetailsManager — reads/writes the users/authorities tables via the standard Spring Security JDBC schema
passwordEncoder	BCryptPasswordEncoder
authenticationManager	Wraps a DaoAuthenticationProvider in a ProviderManager — see correction below
jwtDecoder	NimbusJwtDecoder built from the RSA public key (Section 3) — verifies incoming bearer tokens
jwtEncoder	NimbusJwtEncoder built from the RSA key pair — signs tokens issued at login

Authorization rules: /api/auth/** is permitAll(); /api/admin/** requires the SCOPE_ROLE_ADMIN authority (hasAuthority("SCOPE_ROLE_ADMIN")); everything else just requires authentication. Sessions are stateless (SessionCreationPolicy.STATELESS) — there is no server-side session state, only the bearer JWT on each request.

⚠ Correction: the authenticationManager bean — the actual fix behind the original build error

The scaffold doc (Spring Boot API Gateway Data Scaffold for JWT Auth.docx) shows:

```
@Bean
public AuthenticationManager authenticationManager(JdbcUserDetailsManager userDetailsManager,
        PasswordEncoder passwordEncoder) {
    var provider = new DaoAuthenticationProvider();
    provider.setUserDetailsService(userDetailsManager);
    provider.setPasswordEncoder(passwordEncoder);
    return new ProviderManager(provider);
}
```

This is what shipped in the original plan, and it's what produced the original build error ("SecurityConfig throwing errors attempting to set the provider") this project ran into. The live code instead reads:

```
@Bean
public AuthenticationManager authenticationManager(JdbcUserDetailsManager userDetailsManager,
        PasswordEncoder passwordEncoder, UserDetailsService userDetailsService) {
    var provider = new DaoAuthenticationProvider(userDetailsService);
    provider.setUserDetailsPasswordService(userDetailsManager);
    provider.setPasswordEncoder(passwordEncoder);
    return new ProviderManager(provider);
}
```

What changed, and why it was necessary:

- new DaoAuthenticationProvider() (no-arg) + .setUserDetailsService(...) is the older, setter-based construction pattern. It was deprecated starting in Spring Security 6.3 in favor of constructor injection. On the version

actually in use here — Spring Security 7.1.1 — that older pattern is no longer usable as written, which is the direct cause of the original compile error.

- The fix: a UserDetailsService userDetailsService parameter was added to the bean method (Spring autowires it to the same JdbcUserDetailsManager singleton bean, matched by its UserDetailsService supertype), and passed straight into DaoAuthenticationProvider's constructor: new DaoAuthenticationProvider(userDetailsService).
- provider.setUserDetailsPasswordService(userDetailsManager) was added on top of that. This is a distinct, optional capability — it enables automatic password-hash upgrading on successful login (relevant if a PasswordEncoder's upgradeEncoding() ever indicates an old hash should be re-encoded). JdbcUserDetailsManager implements this interface too, so passing it here was a valid, deliberate enhancement, not a workaround.
- .setPasswordEncoder(passwordEncoder) is unchanged from the original scaffold.

This is flagged specifically because it was an undocumented, hand-made change relative to all three source documents — none of them show or explain this constructor-injection form.

/api/admin/** and the SCOPE_ROLE_ADMIN authority

GET /api/admin/users (Section 6) is gated by:

```
.requestMatchers("/api/admin/**").hasAuthority("SCOPE_ROLE_ADMIN")
```

The authority string looks unusual — worth knowing why, since it's not what you'd guess from the database alone. The authorities table stores the plain value ROLE_ADMIN for an admin user, and JdbcUserDetailsManager loads it verbatim. But TokenService (Appendix A.7) puts that raw authority into a JWT claim named scope, and securityFilterChain's oauth2.jwt(Customizer.withDefaults()) uses Spring Security's default JwtAuthenticationConverter — which reads authorities from a scope/scp claim and prefixes every value with SCOPE_ when building the Authentication object on each request. So the authority actually presented to the authorization check at request time is SCOPE_ROLE_ADMIN, not ROLE_ADMIN — hence the matcher checks for SCOPE_ROLE_ADMIN to match what the token really carries.

Practical implication for adding more role-gated endpoints later: any future hasAuthority(...)/hasRole(...) check against this app's own tokens needs to account for that SCOPE_ prefix, or it will silently never match — see the Troubleshooting reference (Section 9).

6. Domain Layer (DAO / Service / Controller)

This layer matches the original scaffold document exactly — no corrections needed here. Full source in the Appendix; summary:

- Note (A.8) — a record domain model: id, username, title, content.
- NoteDao (A.9) — interface: create, findByUsername, findById, deleteByIdAndUsername.
- JdbcNoteDao (A.10) — the concrete implementation, using Spring's fluent JdbcClient with named parameters and a GeneratedKeyHolder for the auto-generated id.
- NoteService (A.11) — thin @Transactional wrapper around NoteDao; read-only transaction on the fetch path.
- AppController (A.12) — @RestController at /api, exposing the endpoints below.

Endpoint	Behavior
POST /api/auth/register	Creates a user via JdbcUserDetailsManager, BCrypt-encoding the password
POST /api/auth/login	Authenticates via AuthenticationManager, returns a signed JWT via TokenService
GET /api/greet	Authenticated hello-world
GET/POST /api/notes, DELETE /api/notes/{id}	Authenticated CRUD scoped to the caller's own username
GET /api/admin/users	ROLE_ADMIN-only listing of every user (username, enabled), gated by the SCOPE_ROLE_ADMIN authority check (Section 5), via injected JdbcClient.

The controller's constructor also takes a `JdbcClient jdbcClient` parameter (Spring autowires the same auto-configured bean JdbcNoteDao already uses), and defines one more nested DTO record: `UserSummary(String userName, boolean enabled)` — the projection returned by GET /api/admin/users, following the same "DTOs nested inside the controller" convention as AuthRequest/AuthResponse/CreateNoteRequest.

Why not query through JdbcUserDetailsManager instead? Its public API (loadUserByUsername, createUser, updateUser, deleteUser, userExists, changePassword) is scoped entirely to single-user operations — there is no findAllUsers() or equivalent bulk-read method, on this class or its JdbcDaoImpl superclass. Its internal queries are all parameterized by a single username. Querying the users table directly with JdbcClient is the correct tool here, not a workaround — it's the same pattern JdbcNoteDao already uses to read the notes table directly rather than routing through any Spring Security abstraction.

TokenService (A.7) builds the JWT claim set (issuer=self, 1-hour expiry, sub=username, scope=space-joined authorities) and signs it via the injected JwtEncoder.

7. Build & Run

- Ensure PostgreSQL is running locally and app_db_init.sql has been applied (Section 2).
- Ensure src/main/resources/keystore.p12 is the correctly re-keyed file (Section 4), and rebuild so target/classes isn't stale:

```
mvn clean compile
```

- Run via IntelliJ's run configuration for AppBffApplication, or:

```
mvn spring-boot:run
```

- A clean startup ends with the Spring Boot banner and no stack trace, serving on https://localhost:8443.

If startup fails, check Section 9 first — every failure mode encountered while standing this project up is cataloged there with its actual root cause.

8. Verification Walkthrough: Register → Login → Greet

All requests go to https://localhost:8443. /api/auth/** is unauthenticated; everything else requires the bearer token obtained from login.

8.1 Register

```
curl -sk -i -X POST https://localhost:8443/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"admintest"}'
```

Expect 201 Created, body "User registered successfully". A repeat call with the same username returns 409 Conflict.

> **⚠ Critical gotcha — Postman users read this**
>
> The Content-Type header must be exactly application/json. In Postman, this means the Body tab's raw dropdown must be set to JSON, not Text (the default). If it's left on Text (or no Content-Type is sent at all), the request comes back 401 Unauthorized — with the same response shape as hitting a genuinely protected, authenticated-only endpoint with no token at all — even though /api/auth/register is permitAll(). This was verified directly: identical requests, differing only in Content-Type, produced 201 for application/json and 401 for anything else. See Section 9 for the full explanation.

8.2 Login (obtain a token)

```
curl -sk -i -X POST https://localhost:8443/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"admintest"}'
```

Expect 200 OK with a JSON body: {"token": "<JWT>"}.

8.3 Call an authenticated endpoint

```
curl -sk -i https://localhost:8443/api/greet \
  -H "Authorization: Bearer <token from 8.2>"
```

Expect 200 OK, body "Hello, admin".

A successful pass through all three steps confirms: TLS is serving correctly, the datasource and schema are wired correctly, JdbcUserDetailsManager can read/write the users/authorities tables, DaoAuthenticationProvider can authenticate against them, and the RSA-signed JWT round-trips correctly through the resource server's decoder.

9. Troubleshooting & Gotchas Reference

Every failure mode actually hit while standing up this project, with root cause and fix.

Symptom	Root Cause	Fix
UnrecoverableKeyException: ... BadPaddingException: Given final block not properly padded on startup	server.ssl.key-store-password doesn't match the keystore's actual password	Confirm the real password with keytool -list; re-key with keytool -storepasswd if needed (4.4)
Password fix applied but the app still fails to start with the same SSL error	Fix was applied to the wrong copy (e.g. a stray file in ~) or not rebuilt, leaving a stale target/classes copy	Always edit the src/main/resources copy, then rebuild (mvn compile)
Startup fails looking for a key aliased localhost	mkcert always names the PKCS12 alias 1, not the hostname	Set server.ssl.key-alias=1 (or keytool -changealias to match)
mkcert : The term 'mkcert' is not recognized...	Installed via winget in a terminal session that predates the install; PATH not refreshed	Open a new terminal
Bean creation cascades into failure around userDetailsManager/authenticationManager	spring.datasource.* not yet configured, so the DataSource bean can't be created	Set spring.datasource.url/username/password/driver-class-name
relation "users" does not exist at runtime	No schema.sql auto-init is used; schema only exists after manually running app_db_init.sql	psql -U postgres -f app_db_init.sql (Section 2)
Compile error on provider.setUserDetailsService(userDetailsManager) in SecurityConfig	Spring Security 7.1.1 no longer supports the deprecated no-arg-constructor + setter pattern	Constructor-inject instead: new DaoAuthenticationProvider(userDetailsService) (Section 5)
401 Unauthorized on a permitAll() route despite no Authorization header being sent	Request Content-Type was not application/json (Postman raw dropdown left on Text, or omitted)	Set Content-Type: application/json explicitly
keystore.p12 seemingly "vanishes" after running mkcert with a bare filename	mkcert writes to the current working directory; a bare filename lands wherever the shell happened to be	Pass an absolute path to -p12-file, or immediately note/move the file
A new hasRole(...)/hasAuthority("ROLE_X")-gated endpoint returns 403 for a user who does have that role in the authorities table	This app's JWTs carry authorities in a scope claim, and the default JwtAuthenticationConvert	Use hasAuthority("SCOPE_ROLE_X") to match what the token actually carries, following the existing /api/admin/** rule as the template

Symptom	Root Cause	Fix
	er prefixes those with SCOPE_ — so a DB role of ROLE_ADMIN shows up as SCOPE_ROLE_ADMIN on the token, not ROLE_ADMIN (Section 5)	

Appendix A: Full Source Listings

Key — appendix entry, file path, and the guide section that discusses it. Each entry below begins on its own page.

Entry	File	Discussed in
A.1	app-bff/pom.xml	Section 1
A.2	app-bff/src/main/resources/application.properties	Sections 2, 4
A.3	app_db/app_db_init.sql	Section 2
A.4	app-bff/src/main/java/.../AppBffApplication.java	Section 7
A.5	app-bff/src/main/java/.../config/RsaKeyConfig.java	Section 3
A.6	app-bff/src/main/java/.../config/SecurityConfig.java	Section 5
A.7	app-bff/src/main/java/.../service/TokenService.java	Section 6
A.8	app-bff/src/main/java/.../model/Note.java	Section 6
A.9	app-bff/src/main/java/.../dao/NoteDao.java	Section 6
A.10	app-bff/src/main/java/.../dao/JdbcNoteDao.java	Section 6
A.11	app-bff/src/main/java/.../service/NoteService.java	Section 6
A.12	app-bff/src/main/java/.../controller/AppController.java	Sections 5, 6

A.1 pom.xml

app-bff/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-
4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.1.1</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>org.apprenti</groupId>
    <artifactId>app-bff</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name/>
    <description/>
    <url/>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
        <connection/>
        <developerConnection/>
        <tag/>
        <url/>
    </scm>
    <properties>
        <java.version>26</java.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-jdbc</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security-oauth2-resource-server</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc</artifactId>
        </dependency>

        <dependency>
            <groupId>org.postgresql</groupId>
            <artifactId>postgresql</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
```

```
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-jdbc-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security-oauth2-resource-server-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>

</project>
```

A.2 application.properties

app-bff/src/main/resources/application.properties

```
spring.application.name=app-bff

# Server Port
server.port=8443

# SSL/TLS Configuration
server.ssl.enabled=true
server.ssl.key-store-type=PKCS12
server.ssl.key-store=classpath:keystore.p12
server.ssl.key-store-password=#FSISeedsSWDJune2026
server.ssl.key-alias=1

# PostgreSQL Database Configuration

spring.datasource.url=jdbc:postgresql://localhost:5432/app_db
spring.datasource.username=postgres
spring.datasource.password=postgres1
spring.datasource.driver-class-name=org.postgresql.Driver
```

A.3 app_db_init.sql

app_db/app_db_init.sql

```
---------------------------------------
-- Create DB Instance On PSQL Server --
---------------------------------------
-- 0. Must run at command line!
--    psql -U postgres -f app_db_init.sql
-- 1. Run once to initialize BFF SQL
-- 2. Create all Domain Scripts
--     separately
---------------------------------------
-- Created: 08/28/2026
-- Created By: R. Seeds
---------------------------------------

-- Switch context by connecting ( \c ) to admin DB
\c postgres

DROP DATABASE IF EXISTS app_db;
CREATE DATABASE app_db;

-----------------------------------------
-- Initialize Schema for Spring Security --
-----------------------------------------

-- Switch context by connecting ( \c ) to app_db DB
\c app_db

-- Spring Security Auth Schema
CREATE TABLE IF NOT EXISTS users (
    username VARCHAR(50) NOT NULL PRIMARY KEY,
    password VARCHAR(500) NOT NULL,
    enabled BOOLEAN NOT NULL
);

CREATE TABLE IF NOT EXISTS authorities (
    username VARCHAR(50) NOT NULL,
    authority VARCHAR(50) NOT NULL,
    CONSTRAINT fk_authorities_users FOREIGN KEY (username) REFERENCES users(username)
);

CREATE UNIQUE INDEX IF NOT EXISTS ix_auth_username ON authorities (username, authority);

-- Domain Schema Example for Testing or Subsequent Use
CREATE TABLE IF NOT EXISTS notes (
    id BIGSERIAL PRIMARY KEY,
    username VARCHAR(50) NOT NULL,
    title VARCHAR(255) NOT NULL,
    content TEXT NOT NULL
);
```

A.4 AppBffApplication.java

app-bff/src/main/java/org/apprenti/app_bff/AppBffApplication.java

```
package org.apprenti.app_bff;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class AppBffApplication {

    public static void main(String[] args) {
        SpringApplication.run(AppBffApplication.class, args);
    }

}
```

A.5 RsaKeyConfig.java

app-bff/src/main/java/org/apprenti/app_bff/config/RsaKeyConfig.java

```
package org.apprenti.app_bff.config;

import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.interfaces.RSAPrivateKey;
import java.security.interfaces.RSAPublicKey;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class RsaKeyConfig {

    @Bean
    public KeyPair keyPair() {
        try {
            var generator = KeyPairGenerator.getInstance("RSA");
            generator.initialize(2048);
            return generator.generateKeyPair();
        } catch (Exception e) {
            throw new IllegalStateException("Failed to generate RSA KeyPair", e);
        }
    }

    @Bean
    public RSAPublicKey publicKey(KeyPair keyPair) {
        return (RSAPublicKey) keyPair.getPublic();
    }

    @Bean
    public RSAPrivateKey privateKey(KeyPair keyPair) {
        return (RSAPrivateKey) keyPair.getPrivate();
    }
}
```

A.6 SecurityConfig.java

app-bff/src/main/java/org/apprenti/app_bff/config/SecurityConfig.java

```
package org.apprenti.app_bff.config;

import com.nimbusds.jose.jwk.JWKSet;
import com.nimbusds.jose.jwk.RSAKey;
import com.nimbusds.jose.jwk.source.ImmutableJWKSet;
import com.nimbusds.jose.jwk.source.JWKSource;
import com.nimbusds.jose.proc.SecurityContext;
import java.security.interfaces.RSAPrivateKey;
import java.security.interfaces.RSAPublicKey;
import javax.sql.DataSource;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.ProviderManager;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.oauth2.jwt.JwtDecoder;
import org.springframework.security.oauth2.jwt.JwtEncoder;
import org.springframework.security.oauth2.jwt.NimbusJwtDecoder;
import org.springframework.security.oauth2.jwt.NimbusJwtEncoder;
import org.springframework.security.provisioning.JdbcUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        return http
                .csrf(AbstractHttpConfigurer::disable)
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers("/api/auth/**").permitAll()
                        .requestMatchers("/api/admin/**").hasAuthority("SCOPE_ROLE_ADMIN")
                        .anyRequest().authenticated()
                )
                .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .oauth2ResourceServer(oauth2 -> oauth2.jwt(Customizer.withDefaults()))
                .build();
    }

    @Bean
    public JdbcUserDetailsManager userDetailsManager(DataSource dataSource) {
        return new JdbcUserDetailsManager(dataSource);
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
```

```
    }

    @Bean
    public AuthenticationManager authenticationManager(JdbcUserDetailsManager userDetailsManager,
PasswordEncoder passwordEncoder, UserDetailsService userDetailsService) {
        var provider = new DaoAuthenticationProvider(userDetailsService);
        provider.setUserDetailsPasswordService(userDetailsManager);
        provider.setPasswordEncoder(passwordEncoder);
        return new ProviderManager(provider);
    }

    @Bean
    public JwtDecoder jwtDecoder(RSAPublicKey publicKey) {
        return NimbusJwtDecoder.withPublicKey(publicKey).build();
    }

    @Bean
    public JwtEncoder jwtEncoder(RSAPublicKey publicKey, RSAPrivateKey privateKey) {
        var jwk = new RSAKey.Builder(publicKey).privateKey(privateKey).build();
        JWKSource<SecurityContext> jwks = new ImmutableJWKSet<>(new JWKSet(jwk));
        return new NimbusJwtEncoder(jwks);
    }
}
```

A.7 TokenService.java

app-bff/src/main/java/org/apprenti/app_bff/service/TokenService.java

```
package org.apprenti.app_bff.service;

import java.time.Instant;
import java.time.temporal.ChronoUnit;
import java.util.stream.Collectors;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.oauth2.jwt.JwtClaimsSet;
import org.springframework.security.oauth2.jwt.JwtEncoder;
import org.springframework.security.oauth2.jwt.JwtEncoderParameters;
import org.springframework.stereotype.Service;

@Service
public class TokenService {

    private final JwtEncoder encoder;

    public TokenService(JwtEncoder encoder) {
        this.encoder = encoder;
    }

    public String generateToken(Authentication authentication) {
        Instant now = Instant.now();
        String scope = authentication.getAuthorities().stream()
                .map(GrantedAuthority::getAuthority)
                .collect(Collectors.joining(" "));

        JwtClaimsSet claims = JwtClaimsSet.builder()
                .issuer("self")
                .issuedAt(now)
                .expiresAt(now.plus(1, ChronoUnit.HOURS))
                .subject(authentication.getName())
                .claim("scope", scope)
                .build();

        return this.encoder.encode(JwtEncoderParameters.from(claims)).getTokenValue();
    }
}
```

A.8 Note.java

app-bff/src/main/java/org/apprenti/app_bff/model/Note.java

```
package org.apprenti.app_bff.model;

public record Note(Long id, String username, String title, String content) {}
```

A.9 NoteDao.java

app-bff/src/main/java/org/apprenti/app_bff/dao/NoteDao.java

```
package org.apprenti.app_bff.dao;

import org.apprenti.app_bff.model.Note;
import java.util.List;
import java.util.Optional;

public interface NoteDao {
    Note create(Note note);
    List<Note> findByUsername(String username);
    Optional<Note> findById(Long id);
    boolean deleteByIdAndUsername(Long id, String username);
}
```

A.10 JdbcNoteDao.java

app-bff/src/main/java/org/apprenti/app_bff/dao/JdbcNoteDao.java

```
package org.apprenti.app_bff.dao;

import org.apprenti.app_bff.model.Note;
import java.util.List;
import java.util.Optional;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.jdbc.support.GeneratedKeyHolder;
import org.springframework.stereotype.Repository;

@Repository
public class JdbcNoteDao implements NoteDao {

    private final JdbcClient jdbcClient;

    public JdbcNoteDao(JdbcClient jdbcClient) {
        this.jdbcClient = jdbcClient;
    }

    @Override
    public Note create(Note note) {
        var keyHolder = new GeneratedKeyHolder();
        var sql = "INSERT INTO notes (username, title, content) VALUES (:username, :title, :content)";

        jdbcClient.sql(sql)
                .param("username", note.username())
                .param("title", note.title())
                .param("content", note.content())
                .update(keyHolder, "id");

        Number generatedId = keyHolder.getKey();
        Long id = (generatedId != null) ? generatedId.longValue() : null;
        return new Note(id, note.username(), note.title(), note.content());
    }

    @Override
    public List<Note> findByUsername(String username) {
        var sql = "SELECT id, username, title, content FROM notes WHERE username = :username";
        return jdbcClient.sql(sql)
                .param("username", username)
                .query(Note.class)
                .list();
    }

    @Override
    public Optional<Note> findById(Long id) {
        var sql = "SELECT id, username, title, content FROM notes WHERE id = :id";
        return jdbcClient.sql(sql)
                .param("id", id)
                .query(Note.class)
                .optional();
    }

    @Override
    public boolean deleteByIdAndUsername(Long id, String username) {
        var sql = "DELETE FROM notes WHERE id = :id AND username = :username";
```

```
        int rowsAffected = jdbcClient.sql(sql)
                .param("id", id)
                .param("username", username)
                .update();
        return rowsAffected > 0;
    }
}
```

A.11 NoteService.java

app-bff/src/main/java/org/apprenti/app_bff/service/NoteService.java

```
package org.apprenti.app_bff.service;

import org.apprenti.app_bff.dao.NoteDao;
import org.apprenti.app_bff.model.Note;
import java.util.List;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
@Transactional
public class NoteService {

    private final NoteDao noteDao;

    public NoteService(NoteDao noteDao) {
        this.noteDao = noteDao;
    }

    public Note createNote(String username, String title, String content) {
        return noteDao.create(new Note(null, username, title, content));
    }

    @Transactional(readOnly = true)
    public List<Note> getNotesForUser(String username) {
        return noteDao.findByUsername(username);
    }

    public boolean deleteNote(Long id, String username) {
        return noteDao.deleteByIdAndUsername(id, username);
    }
}
```

A.12 AppController.java

app-bff/src/main/java/org/apprenti/app_bff/controller/AppController.java

```
package org.apprenti.app_bff.controller;

import org.apprenti.app_bff.model.Note;
import org.apprenti.app_bff.service.NoteService;
import org.apprenti.app_bff.service.TokenService;
import java.util.List;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.JdbcUserDetailsManager;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api")
public class AppController {

    private final AuthenticationManager authManager;
    private final TokenService tokenService;
    private final JdbcUserDetailsManager userDetailsManager;
    private final PasswordEncoder passwordEncoder;
    private final NoteService noteService;
    private final JdbcClient jdbcClient;

    public AppController(AuthenticationManager authManager,
                         TokenService tokenService,
                         JdbcUserDetailsManager userDetailsManager,
                         PasswordEncoder passwordEncoder,
                         NoteService noteService,
                         JdbcClient jdbcClient) {
        this.authManager = authManager;
        this.tokenService = tokenService;
        this.userDetailsManager = userDetailsManager;
        this.passwordEncoder = passwordEncoder;
        this.noteService = noteService;
        this.jdbcClient = jdbcClient;
    }

    // Records for DTOs
    public record AuthRequest(String username, String password) {}
    public record AuthResponse(String token) {}
    public record CreateNoteRequest(String title, String content) {}
    public record UserSummary(String userName, boolean enabled) {};

    // --- Authentication Endpoints ---

    @PostMapping("/auth/register")
    public ResponseEntity<String> register(@RequestBody AuthRequest req) {
        if (userDetailsManager.userExists(req.username())) {
            return ResponseEntity.status(HttpStatus.CONFLICT).body("Username already exists");
```

```
        }

        var user = User.builder()
                .username(req.username())
                .password(passwordEncoder.encode(req.password()))
                .roles("USER")
                .build();

        userDetailsManager.createUser(user);
        return ResponseEntity.status(HttpStatus.CREATED).body("User registered successfully");
    }

    @PostMapping("/auth/login")
    public ResponseEntity<AuthResponse> login(@RequestBody AuthRequest req) {
        Authentication auth = authManager.authenticate(
                new UsernamePasswordAuthenticationToken(req.username(), req.password())
        );
        String token = tokenService.generateToken(auth);
        return ResponseEntity.ok(new AuthResponse(token));
    }

    // --- Authenticated Hello Endpoint ---

    @GetMapping("/greet")
    public ResponseEntity<String> greet(Authentication authentication) {
        return ResponseEntity.ok("Hello, " + authentication.getName());
    }

    // --- Authenticated Domain Endpoints ---

    @GetMapping("/notes")
    public ResponseEntity<List<Note>> getUserNotes(Authentication authentication) {
        return ResponseEntity.ok(noteService.getNotesForUser(authentication.getName()));
    }

    @PostMapping("/notes")
    public ResponseEntity<Note> createNote(@RequestBody CreateNoteRequest req, Authentication
authentication) {
        Note createdNote = noteService.createNote(authentication.getName(), req.title(),
req.content());
        return ResponseEntity.status(HttpStatus.CREATED).body(createdNote);
    }

    @DeleteMapping("/notes/{id}")
    public ResponseEntity<Void> deleteNote(@PathVariable Long id, Authentication authentication) {
        boolean deleted = noteService.deleteNote(id, authentication.getName());
        return deleted ? ResponseEntity.noContent().build() : ResponseEntity.notFound().build();
    }

    @GetMapping("/admin/users")
    public List<UserSummary> getAllUsers(){
        return jdbcClient.sql("SELECT username, enabled FROM users").query(UserSummary.class).list();
    }
}
```